BBC News Classification

In this assignment, we will use data from https://www.kaggle.com/c/learn-ai-bbc/overview, which is a Kaggle competition is about categorizing news articles. You will use matrix factorization to predict the category, submit your results to Kaggle for test evaluation.

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import time
from collections import Counter
from sklearn.metrics import accuracy_score
from sklearn.decomposition import NMF
import os
```

In [2]:
```
# Loading train data
articles = pd.read_csv("data/bbc/train.csv")
```

In [3]:
```
# Displaying what the data looks like
# It has article id, article texts, and category. Here, category is the laab
articles
```

Out[3]:

	ArticleId	Text	Category
0	1833	worldcom ex-boss launches defence lawyers defe...	business
1	154	german business confidence slides german busin...	business
2	1101	bbc poll indicates economic gloom citizens in ...	business
3	1976	lifestyle governs mobile choice faster bett...	tech
4	917	enron bosses in $168m payout eighteen former e...	business
...	...	...	...
1485	857	double eviction from big brother model caprice...	entertainment
1486	325	dj double act revamp chart show dj duo jk and ...	entertainment
1487	1590	weak dollar hits reuters revenues at media gro...	business
1488	1587	apple ipod family expands market apple has exp...	tech
1489	538	santy worm makes unwelcome visit thousands of ...	tech

1490 rows × 3 columns

In [4]:
```
# Let's print out a sample article text. You'll also see special characters.
articles['Text'].iloc[0]
```

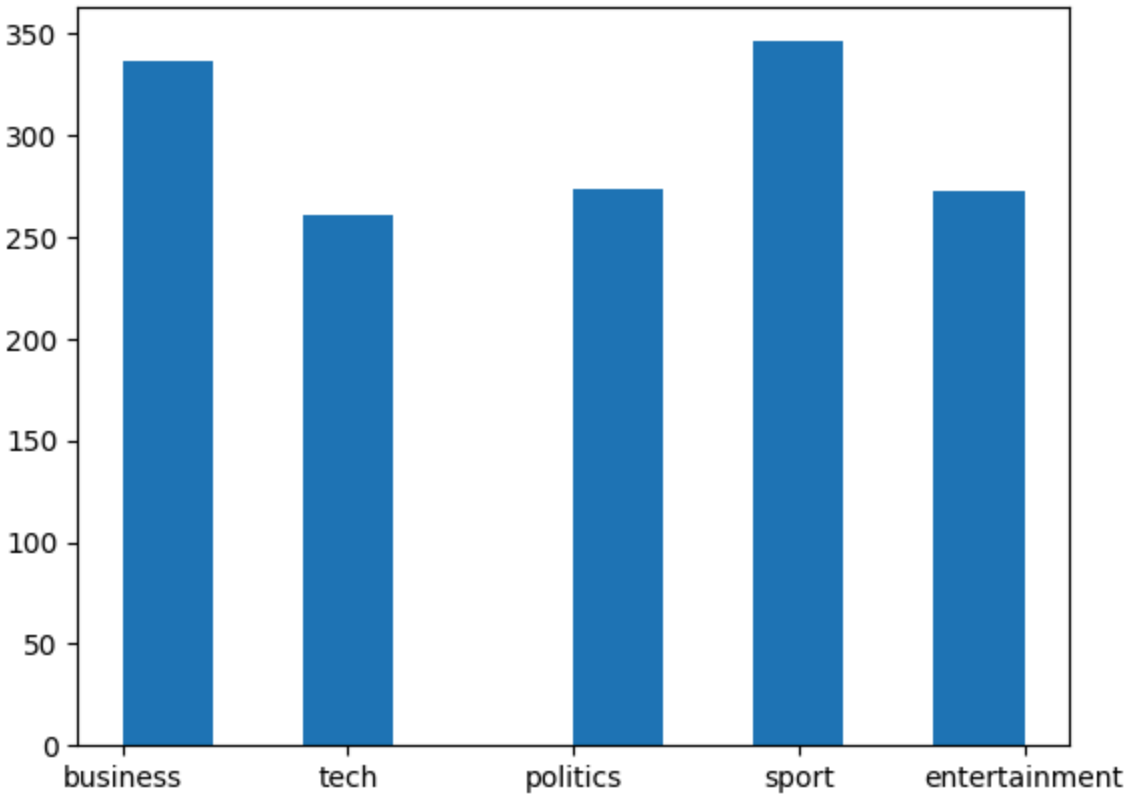
Out[4]:
```
'worldcom ex-boss launches defence lawyers defending former worldcom chief
bernie ebbers against a battery of fraud charges have called a company whis
tleblower as their first witness.  cynthia cooper  worldcom s ex-head of in
ternal accounting  alerted directors to irregular accounting practices at t
he us telecoms giant in 2002. her warnings led to the collapse of the firm
following the discovery of an $11bn (£5.7bn) accounting fraud. mr ebbers ha
s pleaded not guilty to charges of fraud and conspiracy.  prosecution lawye
rs have argued that mr ebbers orchestrated a series of accounting tricks at
worldcom  ordering employees to hide expenses and inflate revenues to meet
wall street earnings estimates. but ms cooper  who now runs her own consult
ing business  told a jury in new york on wednesday that external auditors a
rthur andersen had approved worldcom s accounting in early 2001 and 2002. s
he said andersen had given a  green light  to the procedures and practices
used by worldcom. mr ebber s lawyers have said he was unaware of the fraud
arguing that auditors did not alert him to any problems.  ms cooper also sa
id that during shareholder meetings mr ebbers often passed over technical q
uestions to the company s finance chief  giving only  brief  answers himsel
f. the prosecution s star witness  former worldcom financial chief scott su
llivan  has said that mr ebbers ordered accounting adjustments at the firm
telling him to  hit our books . however  ms cooper said mr sullivan had not
mentioned  anything uncomfortable  about worldcom s accounting during a 200
1 audit committee meeting. mr ebbers could face a jail sentence of 85 years
if convicted of all the charges he is facing. worldcom emerged from bankrup
tcy protection in 2004  and is now known as mci. last week  mci agreed to a
buyout by verizon communications in a deal valued at $6.75bn.'
```

In [5]:
```
# There are 5 unique categories (labels)
articles['Category'].unique()
```

Out[5]:
```
array(['business', 'tech', 'politics', 'sport', 'entertainment'],
      dtype=object)
```

In [6]:
```
# Let's see how many articles are in each category. It shows that the catego
plt.hist(articles['Category'])
```

Out[6]:
```
(array([336.,   0., 261.,   0.,   0., 274.,   0., 346.,   0., 273.]),
 array([0. , 0.4, 0.8, 1.2, 1.6, 2. , 2.4, 2.8, 3.2, 3.6, 4. ]),
 <BarContainer object of 10 artists>)
```



Q1. Text preprocessing

As we have done simple EDA, now, let's extract some feaures from the text. Read sklearn document for TfidfVectorizer and CountVectorizer. Search online about text preprocessing methods based on word count and TF-IDF.

[10 pts] Summarize/explain what those methods are. Why is TF-IDF better than simple word count?

(optional: Search and explain other text preprocessing methods such as GloVe or Word2Vec. you can include python snippets on how to use them).

Word count is simple, intuitive, and easy to implement. However, it doesn't account for the importance of a word across documents, common words may dominate the representation, and results are in sparse, high dimensional vectors. TF-IDF scales the word count by how unique or informative a word is across documents. It reduces the weight of common words, highlights important, rare words, and often improves performance for classification tasks.

In the below example, we will show how to use feature extraction vectorizer. We will show CountVectorizer, but the usage of TfidfVectorizer is also similar. The vectorizer has many options, but `max_features` is most often used. A collection of all words in the all articles is called vocabulary. Since the vocabulary can include so many words, often

we want to limit the number of vocabularies in our feature vector. `max_features` is a parameter that sets the limit.

In [7]:
```
#There are 1490 articles in the train data
data_samples = articles['Text']
print(len(data_samples))
```

```
1490
```

In [8]:
```
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
```

In [9]:
```
# Let's take an example of CountVectorizer
n_features = 200 # For example, we decided to include only 200 words in our
count_vectorizer = CountVectorizer(max_features=n_features, stop_words="engl
```

In [10]:
```
# .fit_transform() transforms the text data to feature vectors.
# Here, the feature matrix from CountVectorizer is a word count vector
wordcount = count_vectorizer.fit_transform(data_samples)
```

In [11]:
```
# This feature matrix has a shape of (# of articles, # max features)
# The matrix is a sparse matrix format for computation efficiency.
wordcount
```

Out[11]:
```
<1490x200 sparse matrix of type '<class 'numpy.int64'>'
        with 40735 stored elements in Compressed Sparse Row format>
```

In [12]:
```
# Although you can convert a sparse matrix to dense matrix to see the conten
# usually we don't want to load a dense matrix of very big matrix (such as a
# Here, just to show what's inside:
wordcount.todense()
# It shows a word count fewature matrix.
```

Out[12]:
```
matrix([[0, 0, 0, ..., 0, 0, 1],
        [0, 1, 0, ..., 0, 1, 0],
        [1, 0, 0, ..., 9, 0, 1],
        ...,
        [0, 1, 1, ..., 0, 7, 0],
        [0, 2, 1, ..., 0, 1, 0],
        [1, 0, 0, ..., 0, 0, 0]])
```

In [13]:
```
#For example, the feature vector for the last third article is this:
wordcount.todense()[-3]
# It has 0 count the first vocabulary, 1 count for the second vocabulary, 1
```

Out[13]:
```
matrix([[0, 1, 1, 0, 2, 3, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 1, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
         0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 2, 2, 0, 0, 0,
         0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
         0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 1, 5, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0,
         0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 7, 0]])
```

In [14]:
```
# .vocabulary_ shows word count for each word in the selected vocabulary in
count_vectorizer.vocabulary_
```

Out[14]:

```
{'chief': 29,
 'called': 26,
 'company': 34,
 'firm': 65,
 'mr': 122,
 'business': 25,
 'told': 179,
 'new': 128,
 'early': 50,
 'said': 155,
 'given': 73,
 'used': 185,
 'did': 46,
 'star': 170,
 'financial': 64,
 'hit': 85,
 'years': 199,
 '2004': 5,
 'week': 192,
 'deal': 43,
 'europe': 57,
 'economy': 52,
 'based': 15,
 'research': 152,
 'january': 94,
 'months': 121,
 'economic': 51,
 'bank': 14,
 'labour': 97,
 'minister': 117,
 'despite': 45,
 'high': 84,
 'expected': 59,
 'year': 198,
 '2003': 4,
 'growth': 80,
 'president': 144,
 'going': 74,
 'half': 81,
 'record': 150,
 'making': 110,
 '10': 1,
 'firms': 66,
 'including': 88,
 'figures': 61,
 'rise': 154,
 'european': 58,
 'bbc': 16,
 'nations': 125,
 'world': 197,
 'service': 164,
 'national': 124,
 'countries': 36,
 'future': 69,
 '000': 0,
 'people': 138,
```

```
 'country': 37,
 'international': 91,
 'just': 95,
 'director': 48,
 'saying': 158,
 'place': 140,
 'far': 60,
 'says': 159,
 'china': 30,
 'seen': 163,
 'government': 77,
 'south': 168,
 'biggest': 20,
 '2005': 6,
 'took': 180,
 'mobile': 118,
 'better': 18,
 'help': 83,
 'phone': 139,
 'technology': 176,
 'industry': 89,
 'michael': 114,
 'media': 113,
 'news': 129,
 'use': 184,
 'way': 191,
 'good': 75,
 'make': 109,
 'start': 171,
 'users': 186,
 'end': 54,
 'able': 7,
 'number': 130,
 'month': 120,
 'taking': 173,
 'uk': 182,
 'film': 62,
 'digital': 47,
 'life': 101,
 'pay': 137,
 'went': 193,
 'public': 146,
 'action': 9,
 'added': 10,
 'group': 79,
 'long': 106,
 'big': 19,
 'lost': 107,
 'howard': 87,
 'play': 142,
 'time': 178,
 'left': 100,
 'don': 49,
 'think': 177,
 'win': 194,
 'days': 42,
```

```
 'game': 70,
 'won': 195,
 'work': 196,
 'like': 102,
 'wales': 189,
 'second': 160,
 'france': 68,
 've': 187,
 'come': 32,
 'ireland': 93,
 'campaign': 28,
 'right': 153,
 'set': 166,
 'really': 148,
 'hard': 82,
 'games': 71,
 'got': 76,
 'british': 23,
 'great': 78,
 'britain': 22,
 'strong': 172,
 'market': 111,
 'sales': 156,
 'million': 116,
 'net': 127,
 'office': 132,
 'day': 41,
 'according': 8,
 '12': 2,
 'blair': 21,
 'prime': 145,
 'general': 72,
 'say': 157,
 'security': 162,
 'plans': 141,
 'law': 99,
 'home': 86,
 'need': 126,
 'want': 190,
 'tv': 181,
 '20': 3,
 'recent': 149,
 'final': 63,
 'tax': 174,
 'open': 135,
 'later': 98,
 'best': 17,
 'team': 175,
 'came': 27,
 'line': 104,
 'club': 31,
 'old': 133,
 'cup': 39,
 'players': 143,
 'awards': 12,
 'music': 123,
```

```
 'radio': 147,
 'know': 96,
 'court': 38,
 'lot': 108,
 'computer': 35,
 'match': 112,
 'money': 119,
 'london': 105,
 'united': 183,
 'information': 90,
 'internet': 92,
 'services': 165,
 'offer': 131,
 'online': 134,
 'decision': 44,
 'companies': 33,
 'software': 167,
 'away': 13,
 'anti': 11,
 'microsoft': 115,
 'eu': 56,
 'foreign': 67,
 'secretary': 161,
 'spokesman': 169,
 'report': 151,
 'brown': 24,
 'england': 55,
 'video': 188,
 'party': 136,
 'data': 40,
 'election': 53,
 'likely': 103}
```

Q2. Topic Modeling using NMF

Below are the starter codes. We will build TopicModeling class which predicts article labels using NMF algorithm. You'll need to complete `factorize` and `predict` methods.

Q2.a Complete factorize function [15 pts]

5 pts each for each STEP. You can do the similar fro mthe example above.
Note: `self.X` is the train and test data combined. In the STEP2 in `factorize` function, we fit all the data so that the feature matrix contains vocabularies from both trin and test data. You could fit with only texts from train data, but then there might be some vocabularies from test data that don't exist in train data. Since we're not showing the labels to the model, using the vectorizer feature extraction with combined data is ok.

Q2.b Complete STEP4 in predict function [10 pts]

self.features is a feature matrix from the tf-idf vectorizer. You can retrieve the fitted feature matrix from train data portion by `self.features[:self.n_tr]` You can calculate `yp_tr` using this train part of the feature matrix. Use the matrix factorization formula (theory) for prediction. It involves some dot products and transpose of matrices. numpy operations require matrices to be numpy array format, which is why we reformatted `tfidf` to `self.features` using `numpy.array` at the end of the `factorize` function.

Q2.c Complete STEP 5 in predict function [15 pts]

Map the numeric label values 0,1,2,3,4 from the prediction to category strings. Save the test prediction labels to `self.test['Category']`. For example, `self.test['Category']` may look like:

0	sport
1	tech
2	sport
3	business
4	sport
...	
730	business
731	entertainment
732	tech
733	business
734	politics

Name: Category, Length: 735, dtype: object

How can you map the integers to the string labels? You can use train data: You can compare the train label string `yt[i]` and the predicted integers from train data `yp_tr[i]` for each sample. Since the model isn't going to predict on train data perfectly, sometimes the match may be wrong. But if you keep track of the matching predicted integer labels for each string label then you can find the majority of the integer index corresponding to the string label.

In [24]:
```
class TopicModeling():
    def __init__(self):
        self.getdata()

    def getdata(self):
        self.train = pd.read_csv("data/bbc/train.csv")
        self.test = pd.read_csv("data/bbc/test.csv")
```

```
        self.X = list(self.train.Text)+list(self.test.Text) # To make the co
        self.n_tr = len(self.train)
        self.n_te = len(self.test)

    def factorize(self,n_features):
        self.n_features = n_features
        # STEP1. Construct tf-idf vectorizer that accepts max features of n_
        tfidf_vectorizer = TfidfVectorizer(max_features=n_features, max_df=0
        # YOUR CODE HERE


        # STEP2. Fit the tfidf_vectorizer using the all data X above. Assign
        tfidf = tfidf_vectorizer.fit_transform(self.X)
        # YOUR CODE HERE


        # STEP3. Fit the model using sklearn NMF and assign to self.model
        self.model = NMF(n_components=5, random_state=42)
        self.W = self.model.fit_transform(tfidf)
        self.H = self.model.components_
        # YOUR CODE HERE

        self.tfidf = tfidf
        self.features = np.array(tfidf.toarray()) #saves the feature matrix

    def predict(self):
        # STEP4. Predict labels for train and test data using matrix algebra
        # Refer to the usage in https://scikit-learn.org/stable/modules/gene
        # The predicted labels are numeric; 0-4
        W_train = self.model.transform(self.features[:self.n_tr])
        W_test = self.model.transform(self.features[self.n_tr:])
        yp_tr = np.argmax(W_train, axis=1) # prediction from train data
        yp_te = np.argmax(W_test, axis=1) # prediction from test data
        # YOUR CODE HERE


        # STEP5. Map the numeric values 0-4 from the prediction to string va
        # You can compare the true labels from train data with yp_tr predict
        # Then you know which number label in yp_tr corresponds which string
        # update self.test['Category'] to the string labels accordingly.
        yt = list(self.train['Category'])

        # create mapping from topic index to category name using majority vo
        label_map = {}
        topic_to_labels = {}
        for i in range(len(yt)):
            topic = yp_tr[i]
            label = yt[i]
            if topic not in topic_to_labels:
                topic_to_labels[topic] = []
            topic_to_labels[topic].append(label)
        for topic, labels in topic_to_labels.items():
            most_common_label = Counter(labels).most_common(1)[0][0]
            label_map[topic] = most_common_label

        # apply mapping to train and test predictions
```

```
        mapped_train_preds = [label_map[i] for i in yp_tr]
        mapped_test_preds = [label_map[i] for i in yp_te]

        # calculate and print train accuracy
        train_acc = accuracy_score(yt, mapped_train_preds)
        print(f"Train Accuracy for n_features = {self.n_features}: {train_ac

        self.test['Category'] = mapped_test_preds

        # YOUR CODE HERE


    def save(self): # This function helps to create submission file
        if not os.path.isdir('submission'):
            os.mkdir('submission')
        self.test[["ArticleId","Category"]].to_csv("submission/submission_f"
```

Q3. Tune hyper parameters [10 pts]

Change your n_features (for example, between 1000 to 10000). Run prediction which will predict and save Print the train accuracy for each n feature. (You can add print statement for train accuracy in the predict function above) Save the test prediction using save function above.

In [25]:
```
# YOUR CODE HERE
feature_list = [1000, 2000, 3000, 5000, 7000, 10000]
results = []

for n_feat in feature_list:
    print(f"--- Tuning for n_features = {n_feat} ---")
    model = TopicModeling()
    model.factorize(n_features=n_feat)
    model.predict()
    model.save()
```

```
--- Tuning for n_features = 1000 ---
Train Accuracy for n_features = 1000: 0.8671
--- Tuning for n_features = 2000 ---
Train Accuracy for n_features = 2000: 0.8886
--- Tuning for n_features = 3000 ---
Train Accuracy for n_features = 3000: 0.8993
--- Tuning for n_features = 5000 ---
Train Accuracy for n_features = 5000: 0.8946
--- Tuning for n_features = 7000 ---
Train Accuracy for n_features = 7000: 0.8940
--- Tuning for n_features = 10000 ---
Train Accuracy for n_features = 10000: 0.8953
```

Q4. Best results [10 pts]

Submit a few test prediction (judge based on train accuracy, although the best train accuracy doesn't mean the best test accuracy) and pick which n_features led to the

best test acc. Record the result. Discuss your observations.

Kaggle Test Accuracy:

- 1000 features: 0.89387
- 2000 features: 0.89523
- 3000 features: 0.90476
- 5000 features: 0.89387
- 7000 features: 0.89523
- 10000 features: 0.89387

The best n_features is 3000 with a train accuracy of 0.8993 and test accuracy of 0.90476.

I observed that train accuracy generally increased with more features however, the best test performance on Kaggle was at 3000 features. Beyond 3000 features, train accuracy remained stable but test accuracy dropped slightly, possibly due to overfitting. This suggests that using too many features may hurt generalization, even if training accuracy remains high.

In [ ]: